

WILLIAM GALVIN

+1 (705) 446-5576 ♦ Toronto, ON

liam.galvin@bell.net ♦ williamgalvin.me ♦ [LinkedIn](#)

EDUCATION

Honours Bachelor of Computer Science (Mobile Computing), Sheridan College

2023 – Dec. 2027

SKILLS

Languages: C++, Python, Java

Technologies: Linux, kdb+/q, SQL, Polars, CMake, Qt, LLVM, Git, Bash, Docker

Concepts: Low-Latency Systems, Concurrency, Compiler Design, Distributed Systems, Networking

EXPERIENCE

Software Developer Intern, Quantitative Technology Services

May – Aug. 2026

RBC Capital Markets, Central Risk Book

Toronto, ON

- Built trading systems and tooling for the bank's designated market making desk on Canadian equities and ETFs.
- Wrote a fill reconciliation pipeline over two kdb+ tables in Polars, matching fills between the risk system of record and the drop-copied UHF system and isolating one-sided breaks; added an intraday monitor that alerts on discrepancies. Both in production daily.
- Owned build, test, and release for a Qt/C++ trading GUI: restructured **CMake** into a monorepo, added unit tests and a bundled launcher, and scripted automated versioning, pre-release promotion, and rollback.
- Cut compressed trading logs a further 20.9% beyond gzip across a seven-year archive via frequency-selected key-tuple substitution, preserving readability and byte-exact MD5 rehydration.
- Replaced a legacy Java compliance reporter with a centralized DMM securities table split into write and read-only report paths; built an IMAP IDLE inbox monitor with config-driven parsers and alerting on missing or unparseable messages.

Lead Organizer & Founder

November 2025

Sheridan Datathon

Mississauga, ON

- Founded Sheridan's first datathon, themed around UN Sustainability Goals; attracted **200+ participants**.
- Led **30+ organizers** and **9 mentors**; raised **\$6.8k** in sponsorships.
- Managed end-to-end execution: venue, competition design, website, judging, marketing, and registration.

PROJECTS

Low-Latency Benchmarking Engine

C++23, CMake, Python

- Built a benchmarking framework using RDTSC cycle-accurate timing, with a generic `stats::Summary<T>` computing latency distributions (min/max/percentiles/stddev) across runs.
- Benchmarked low-latency data structures including a policy-based ring buffer, measuring per-operation cycle costs under varying configurations.
- Applied modern C++ idioms—CRTP, concepts, `[[nodiscard]]`, RAII, and `std::expected`—with a CMake/Ninja build targeting reproducible, zero-overhead measurement.

Cherry Compiler

C++, LLVM

- Designing a dual-language system: Pit (low-level) and Cherry (high-level), where Cherry compiles to Pit.
- Hand-writing performant lexers and recursive descent parsers for both languages against formally specified grammars.
- Lowering ASTs into a custom intermediate representation and generating LLVM code.